# GIT & GITHUB COMMAND HANDBOOK

A practical reference for the Analytics with Claude Power BI AI Analyst project

| PROJECT DETAILS | VALUE |
|---|---|
| Local project folder | D:\Study\PowerBI Projects\Analytics with Claude |
| GitHub repository | dhanushkapg/AirLine-Performace-Report |
| Primary branch | main |

**Purpose**
**This handbook explains the Git commands used to create, manage, commit and synchronise your local project with GitHub. It also includes a safe daily workflow and troubleshooting guidance.**

Prepared as a reusable learning reference • Windows PowerShell / VS Code

# 1. What happened during your setup

Your local Git repository and GitHub repository initially had separate histories. Git also detected that your local branch was named **main**, while you first attempted to push **master**. Once the branch name and remote history were handled, Git could synchronise the two repositories.

| Message / issue | Meaning | Resolution |
|---|---|---|
| fatal: not a git repository | The folder was not yet a Git repository. | Run git init in the project folder. |
| remote origin already exists | A remote named origin was already configured. | Inspect it with git remote -v; update with git remote set-url. |
| src refspec master does not match any | There was no local master branch/commit to push; your branch was main. | Push main: git push -u origin main. |
| non-fast-forward | GitHub had a commit missing from the local history. | Pull and merge the remote history before pushing. |
| Vim merge message appeared | Git opened an editor to confirm the merge commit message. | Press Esc, type :wq, then press Enter. |

## Key concepts

**Git** manages versions and commit history on your computer. **GitHub** hosts a remote copy of the repository. **Commit** saves a snapshot locally. **Push** uploads local commits. **Pull** downloads and integrates remote changes.

# 2. Basic navigation and project commands

| Command | Description |
|---|---|
| pwd | Displays the current working directory. |
| dir | Lists files and folders in PowerShell. |
| cd "D:\Study\PowerBI Projects\Analytics with Claude" | Moves into the project folder. Quotes are important because the path contains spaces. |
| code . | Opens the current folder in Visual Studio Code. |
| cls | Clears the PowerShell terminal. |

# 3. Initialise and inspect a repository

| Command | Description |
|---|---|
| git init | Creates a new local Git repository in the current folder. |
| git status | Shows the current branch and staged, unstaged or untracked changes. |
| git status --short | Shows a compact status summary. |
| git rev-parse --show-toplevel | Displays the root folder of the Git repository. |
| git rev-parse --is-inside-work-tree | Checks whether the current folder is inside a Git working tree. |
| git ls-files | Lists files currently tracked by Git. |

# 4. Configure your Git identity

Configure this once on your computer. Use the name and email you want associated with your commits.

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
git config --global --list
```

Useful checks: **git config user.name** and **git config user.email** display the identity configured for the current repository (including applicable global settings).

# 5. Stage and commit files

| Command | Description |
|---|---|
| git add . | Stages eligible changes under the current directory. |
| git add -A | Stages additions, modifications and deletions throughout the repository. |
| git add filename.py | Stages one specified file. |
| git diff | Shows changes that have not been staged. |
| git diff --staged | Shows changes that are staged for the next commit. |
| git commit -m "message" | Creates a local commit from staged changes. |

## Example

```
git status
git add .
git diff --staged
git commit -m "Add flight analysis dashboard"
```

**A commit only records changes in your local repository. It does not upload them to GitHub.**

# 6. Branch management

| Command | Description |
|---|---|
| git branch | Lists local branches; the current branch has an asterisk. |
| git branch -a | Lists local branches and remote-tracking branches. |
| git branch -M main | Renames the current branch to main. |
| git switch main | Switches to the main branch. |
| git switch -c feature-name | Creates a new branch and switches to it. |
| git branch -d feature-name | Deletes a local branch that has been merged. |

# 7. Connect to GitHub (remote commands)

| Command | Description |
|---|---|
| git remote -v | Shows the configured remote names and URLs. |
| git remote add origin URL | Adds a remote named origin. |
| git remote set-url origin URL | Changes the URL of an existing origin remote. |
| git remote remove origin | Removes the origin configuration from this local repository; it does not delete the GitHub repository. |

## Your repository remote

```
git remote set-url origin https://github.com/dhanushkapg/AirLine-Performace-Report.git
git remote -v
```

If **remote origin already exists** appears, do not add it again. Check the URL and use **git remote set-url** if it needs correcting.

# 8. Push, pull and fetch

| Command | Description |
| --- | --- |
| git push | Uploads local commits to the configured upstream branch. |
| git push -u origin main | Pushes main and sets origin/main as its upstream for future pushes and pulls. |
| git pull | Fetches and integrates changes from the configured upstream branch. |
| git pull origin main | Fetches and integrates the remote main branch. |
| git fetch | Downloads remote updates but does not merge them into the current branch. |
| git fetch --all | Fetches updates from all configured remotes. |

# 9. Inspect commit history

| Command | Description |
| --- | --- |
| git log | Displays detailed commit history. |
| git log --oneline | Displays a compact one-line history. |
| git log --oneline --graph --all | Shows a compact graph of local and remote-tracking histories. |
| git show | Displays details of the latest commit. |

# 10. Merge and conflict commands

| Command | Description |
|---|---|
| git merge branch-name | Merges the named branch into the current branch. |
| git merge --abort | Aborts an in-progress merge and returns to the pre-merge state where possible. |
| git pull origin main --allow-unrelated-histories | Allows Git to combine histories that began independently, such as a local repository and a GitHub repository that each had an initial commit. |
| git add . | Stages files after you have manually resolved conflicts. |
| git commit -m "Merge remote changes" | Completes a merge if Git requires you to create the merge commit manually. |

## If VS Code shows a merge conflict

Open each conflicted file, review both versions, and decide which content to keep. VS Code may offer **Accept Current Change**, **Accept Incoming Change** or **Accept Both Changes**. Review the final file, save it, then stage the resolved file and complete the merge if required.

## Vim opened for a merge message

```
Press Esc
Type :wq
Press Enter
```

This saves the merge message and exits Vim. To exit without saving, use **:q!** only if you intend to cancel that editor operation.

# 11. Undo and recovery commands

| Command | Description |
|---|---|
| git restore filename | Discards unstaged changes to the specified tracked file. This can lose edits. |
| git restore --staged filename | Unstages a file but keeps its working copy changes. |
| git revert COMMIT_ID | Creates a new commit that reverses the changes introduced by an earlier commit. |
| git reset --soft HEAD~1 | Moves the branch back one commit but keeps changes staged. |
| git reset --mixed HEAD~1 | Moves the branch back one commit and leaves changes unstaged. |
| git stash | Temporarily stores uncommitted tracked changes. |
| git stash pop | Applies the most recent stash and removes it from the stash list if successful. |

**Caution: Avoid git reset --hard and git push --force unless you understand the consequences. They can discard local work or overwrite remote history. Prefer git revert when you need to undo a commit that has already been shared.**

# 12. Clone a repository

To create a local copy of your GitHub repository on another computer or in another folder:

```
git clone https://github.com/dhanushkapg/AirLine-Performace-Report.git
```

Git creates a new folder containing the repository files and history. Open that folder in VS Code with **code .**.

# 13. Recommended .gitignore for this project

Create a file named **.gitignore** in the project root. Review it before staging files. The example below excludes common Python environment files and secrets while allowing an example environment template.

```
.env
.env.*
!.env.example

.venv/
venv/
__pycache__/
*.py[cod]
.ipynb_checkpoints/
*.log
```

Do not blindly ignore all Power BI project files. For this project, version-control appropriate **.pbip**, report-definition and **TMDL** files. Exclude confidential source data, credentials, local caches and any files your organisation does not permit you to publish.

# 14. Your recommended daily workflow

Use this sequence when returning to the project after the first GitHub upload has been completed.

```
# 1. Go to the project folder
cd "D:\Study\PowerBI Projects\Analytics with Claude"

# 2. Open the project in VS Code
code .

# 3. Check the repository
git status

# 4. Get the latest remote changes
git pull

# 5. Work on Python, Power BI and project files

# 6. Review changes
git status
git diff

# 7. Stage changes
git add .

# 8. Review staged changes
git diff --staged

# 9. Commit
git commit -m "Describe your changes"

# 10. Upload commits
git push
```

If there are no changes to commit, Git may report **nothing to commit, working tree clean**. That is normal. If there are no new commits to upload, **git push** may report that everything is up to date.

# 15. Troubleshooting quick reference

| Message | What to check / do |
|---|---|
| fatal: not a git repository | Run pwd and dir. Navigate to the correct project folder, then run git status. Use git init only if this folder truly has no repository. |
| remote origin already exists | Run git remote -v. Correct the existing URL with git remote set-url origin URL. |
| src refspec master does not match any | Run git branch and git log --oneline. Push the branch that actually exists and has a commit, e.g. git push -u origin main. |
| non-fast-forward | The remote has commits missing locally. Inspect status, then integrate remote changes with git pull before pushing. Do not force-push to bypass the warning. |
| nothing to commit, working tree clean | There are no uncommitted changes. Check git log --oneline and git status -sb to inspect commits and upstream tracking. |
| Authentication failed | Complete GitHub sign-in using the browser or your configured Git credential manager. Do not paste passwords or tokens into chat. |

# 16. Final verification checklist

Run these commands to check your local repository and its GitHub connection:

```
git status -sb
```

```
git branch -vv
git remote -v
git log --oneline -5
```

A healthy synchronised repository commonly shows a status similar to:

```
## main...origin/main
```

This means the local main branch is tracking origin/main and has no ahead/behind changes. The full status may also say: **nothing to commit, working tree clean**.

# 17. Security reminders for the Power BI + Claude project

• Never commit Claude or Anthropic API keys, passwords, tokens or private certificates.
• Keep real credentials in a local **.env** file and exclude it using **.gitignore**.
• Consider adding an **.env.example** containing placeholder variable names, never real values.
• Do not publish confidential flight data, customer data or restricted Power BI extracts.
• Review **git status** and **git diff --staged** before every commit.
• If a secret is accidentally committed, removing it in a later commit is not enough: revoke or rotate the exposed credential immediately and follow GitHub's secret-removal guidance.

**Core memory aid**

**git add = stage changes**
**git commit = save a local snapshot**
**git push = upload commits to GitHub**
**git pull = fetch and integrate remote changes**
**git fetch = download remote updates without integrating them**

End of handbook • Analytics with Claude